Security Assurance for Modern Big Data APIs

Enea Manzi[1], Marco Anisetti[1,2]

enea.manzi@unimi.it
https://eneamanzi.github.io/

[1]Department of Computer Science, Università degli Studi di Milano

[2]Moon Cloud srl

ITADATA 2026

Context: Big Data APIs & Security Assurance

- **Big Data Ecosystem:** rely on microservices and distributed pipelines, exposed via REST APIs
- **The Trust Boundary:** API Gateways centralize critical enforcement (authentication, authorization, ...)
- **The Assurance Gap:** Security focuses on infrastructure, while API testing targets functional correctness, leaving the interface layer exposed.
- **The Consequence:** Interface-level vulnerabilities (e.g., cross-tenant access) can compromise the entire Big Data pipeline

State of Art Limits (Motivations) & Objectives

State of the Art Limitations

- **Syntactic Blindness:**
  Checks request/response syntax, ignoring security contract semantics
- **The Oracle Problem:**
  Lacks automated criteria for security guarantee.
- **Unreliable Execution:**
  Non-deterministic outcomes (flakiness) prevent continuous CI/CD integration

Our Objectives

- **Semantic Verification:**
  Move beyond syntax to verify the actual behavior of security controls.
- **Specified Oracles:**
  Automate pass/fail verdicts using hardcoded, domain-derived rules.
- **Agnostic & Deterministic:**
  Build a reproducible engine, portable across any documented target.

Our Approach: Contract-Driven API Assurance

Core Idea: Derive syntactically valid probes from OpenAPI contracts and evaluate security semantics using deterministic, specified oracles.

- **Application Agnosticism & Contract-Driven Probing**
  Zero target-specific code; input generation driven strictly by OpenAPI specs and configuration.
- **Specified Oracles for Security Guarantees**
  Domain-derived evaluation rules to detect semantic violations beyond HTTP status codes.
- **Eight-Domain Taxonomy & Privilege Gradient**
  Structured test coverage scaling with available vantage points (Black, Grey, White Box).
- **Deterministic Assessment Engine**
  Topological scheduling and LIFO teardown ensuring 100% reproducible verdicts for CI/CD.

Approach Details: Agnosticism & Contract-Driven

- **Application Agnosticism:** Zero target-specific code. Logic is driven entirely by `openapi.yaml` and `config.yaml`.
-
- **Configuration as Source of Truth:** All deployment variables (URLs, credentials) are isolated from the engine.
-
- **Contract-Driven Probing:** The specification generates syntactically valid requests and defines the exact attack surface.

Approach Details: Taxonomy & Privilege Gradient

- **8-Domain Taxonomy (D0–D7):** Structured security guarantees (e.g., Auth, RBAC, Integrity) aligned with NIST standards.
-
- **Privilege-Aware Coverage (Box Gradient):** Coverage scales dynamically based on available access:
-
  - *BLACK_BOX:* Network-level perimeter controls.
  - *GREY_BOX:* Multi-role credentials for segregation (RBAC/BOLA).
  - *WHITE_BOX:* Gateway administrative configuration plane.

Approach Details: Specified Oracles

- **Semantic Evaluation:** Moves beyond basic syntax (e.g., HTTP 500 errors) to judge actual security behavior.
-
- **3-Step Oracle Construction:**
  1. Identify the specific security guarantee.
  2. Define its behavioral violation.
  3. Encode the evaluation rule.
- **Epistemic Limit:** Verdicts rely on contract compliance; an inaccurate specification yields flawed assumptions.

Approach Details: Deterministic Engine

- **DAG-Based Scheduling:** Resolves test dependencies statically before execution (e.g., auth before access).
- **100% Reproducible:** Fixed configuration and strict LIFO teardown guarantee identical outcomes across runs.
- **Actionable Output:** Distinct semantic exit codes route results directly to CI/CD pipelines.

CHANGE: Example

CHANGE: l'esempio si può raccontare dentro ogni componente, mentre lo presenti, oppure passo passo dopo l'approccio. Dipende dal tempo.

Experimental Evaluation

Testbed

- **Forgejo** 14.0.3 – **Kong** 3.9 DB-less
- Chosen for its structural properties: native OAS, multi-tenant roles, gateway enforcement

Results

- **Determinism:** 2 independent runs produced byte-identical outcomes
- 18 tests, 7 domains: **9 PASS**, **7 FAIL**, **2 SKIP**, **0 ERROR** – 98 findings

Discrepancy between Contract and Reality

Endpoints marked as protected in the specification, but genuinely public, responded without credentials (Test 1.1).

CHANGE: Results

CHANGE: i risultati, per esempio come grafici, con una sola conclusione per grafico.

Conclusions & Future Work

Conclusions

- Contract-driven testing successfully evaluates security semantics beyond simple syntactic conformance (RQ1).
- Specified oracles provide deterministic, automated pass/fail criteria.
- The engine's reproducibility enables reliable integration into continuous release cycles (RQ2).

Future Work

- Develop adapters for managed cloud API gateways (AWS, Azure, GCP).
- Transition from manually authored rules to automated generation of security oracles.

Thanks! Questions?

Enea Manzi
enea.manzi@unimi.it
https://eneamanzi.github.io/
https://www.linkedin.com/in/eneamanzi